

Práctica 3: Análisis de Renta y Educación en Canarias

Explicación detallada de gráficos ggplot con Dagster

Laura Lasso García

Febrero 2026

Índice general

1. Práctica 3: Calidad de Datos con Asset Checks en Dagster	2
1.1. Configuración de la rama de trabajo	2
1.2. Análisis del fichero <code>test_checks.py</code>	2
1.2.1. ¿Qué hace el asset en el script?	2
1.2.2. ¿Qué verificación se ha hecho?	2
1.3. Análisis del fichero <code>definitions.py</code>	3
1.3.1. ¿Qué assets y checks están en el manifiesto?	3
1.3.2. Lanzamiento del proyecto con <code>definitions.py</code>	3
1.4. Matriz de calidad de checks implementados	4
1.5. Implementación de casos sucios por etapa	5
1.5.1. Etapa de Carga	6
1.5.2. Etapa de Transformación	9
1.5.3. Etapa de Visualización	13

Capítulo 1

Práctica 3: Calidad de Datos con Asset Checks en Dagster

1.1. Configuración de la rama de trabajo

Siguiendo las instrucciones de la práctica, se parte de la rama principal actualizada y se crea una nueva rama específica para la implementación de los checks de calidad:

```
git checkout main
git pull
git checkout -b practica-calidad-checks
```

Una vez finalizados todos los cambios, se realiza el commit y push correspondientes:

```
git add .
git commit -m "Practica 3: implementacion de asset checks de
calidad"
git push origin practica-calidad-checks
```

1.2. Análisis del fichero test_checks.py

1.2.1. ¿Qué hace el asset en el script?

El script `test_checks.py` define un asset llamado `islas_raw` que carga el fichero `codislas.csv` y devuelve un `DataFrame` de `pandas`. En su versión limpia (línea 12 inactiva), el CSV se lee correctamente con separador de coma, obteniendo las columnas `año`, `isla`, `medida` y `valor`.

1.2.2. ¿Qué verificación se ha hecho?

El check asociado `check_islas_columnas` busca duplicados implícitos causados por diferencias de mayúsculas y minúsculas. Primero cuenta cuántos nombres de islas hay tal cual vienen en el archivo, y luego transforma todos los nombres a formato `Capitalize` y vuelve a contar. Si el número de categorías originales es igual al de las normalizadas (pasaría el check), significa que todos los nombres ya vienen escritos de forma idéntica. Si hay más categorías originales que normalizadas (no pasaría el check), indica que hay basura en los datos (por ejemplo, `'Lanzarote'` y `'lanzarote'`).

Esto se realiza para evitar la fragmentación visual (principio de Gestalt de Similitud); si los datos no están estandarizados, una herramienta como plotnine interpretará que 'Tenerife' y 'tenerife' son dos islas diferentes. Por ello, el gráfico mostraría dos barras separadas para la misma isla.

1.3. Análisis del fichero definitions.py

1.3.1. ¿Qué assets y checks están en el manifiesto?

El manifiesto central contenido en el archivo definitions.py integra todos los elementos del módulo test_checks mediante cargadores automáticos de Dagster. En cuanto a los activos, el manifiesto incluye todos aquellos decorados con @asset dentro de dicho módulo, destacando específicamente el asset islas_raw, que actúa como el componente principal de datos en este ejemplo. Respecto a los controles de calidad, el objeto Definitions registra todos los checks definidos en el mismo archivo gracias a la función load_asset_checks_from_modules, lo que incorpora formalmente el check de estandarización denominado check_estandarizacion_islas.

1.3.2. Lanzamiento del proyecto con definitions.py

```
dagster dev -f definitions.py
```

Una vez lanzado, se deben materializar los assets que haya y éstos pasarán por los checks que correspondan. En este caso, sólo se materializaría el asset raw_islas y pasaría el check check_estandarizacion_islas. En la imagen 1.1 se observa cómo el asset fue materializado y pasó el check correctamente.

The screenshot displays the Dagster web interface for the asset 'islas_raw'. The 'Checks' tab is active, showing a list of checks on the left and details for 'check_estandarizacion_islas' on the right. The check is marked as 'Succeeded'. The 'Latest execution' table shows a 'Passed' result at '20 feb, 17:29'. The 'Metadata' section provides additional context.

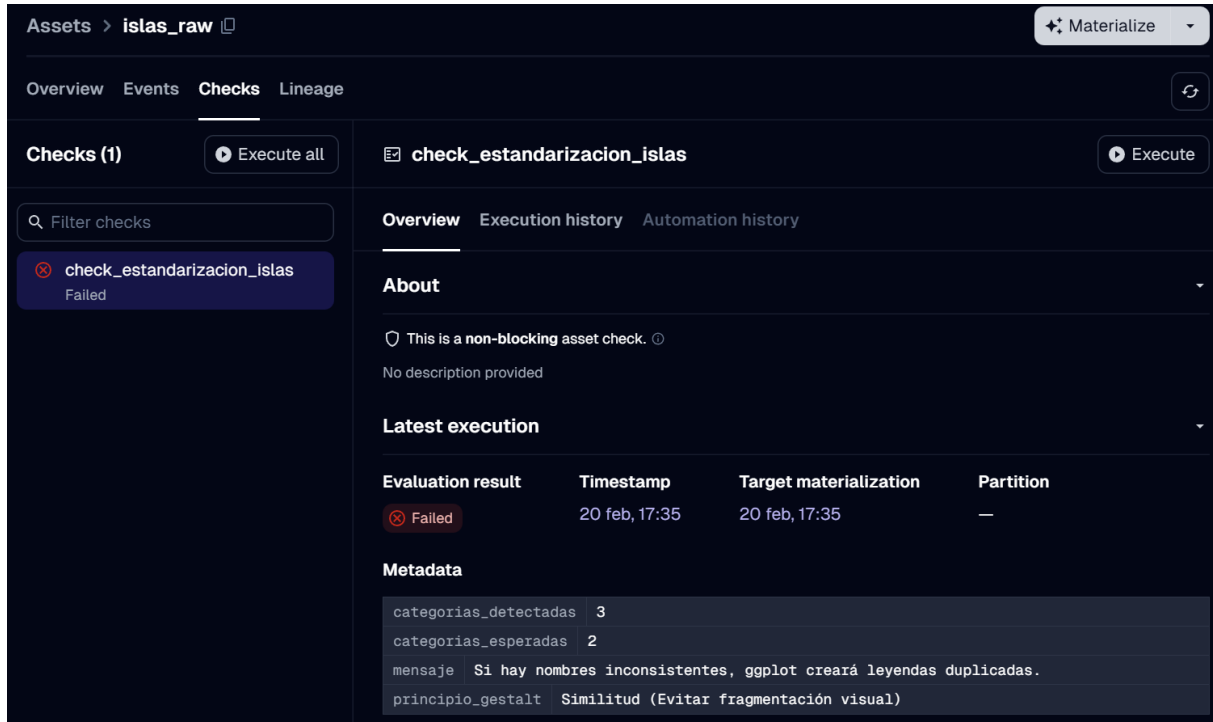
Evaluation result	Timestamp	Target materialization	Partition
Passed	20 feb, 17:29	20 feb, 17:29	—

Metadata

categorias_detectadas	2
categorias_esperadas	2
mensaje	Si hay nombres inconsistentes, ggplot creará leyendas duplicadas.
principio_gestalt	Similitud (Evitar fragmentación visual)

Figura 1.1: Resultado de la materialización con el check pasado correctamente.

Luego, se debe descomentar la línea 12 del archivo `test_checks.py` con el fin de añadir una fila sucia para forzar el fallo del check. Esta fila sucia incluye en la isla 'tenerife', para así al comparar las islas originales detectadas con las normalizadas el número no sea el mismo. Como se observa en la imagen 1.2, una vez materializado este asset con la fila sucia, se observa que no pasa el check.



The screenshot shows the Databricks interface for the 'islas_raw' asset. The 'Checks' tab is active, showing a list of checks on the left and details for 'check_estandarizacion_islas' on the right. The check has failed, indicated by a red 'X' icon and the word 'Failed'.

Checks (1) [Execute all]

- check_estandarizacion_islas [Execute]

check_estandarizacion_islas

Overview Execution history Automation history

About

This is a non-blocking asset check. ⓘ

No description provided

Latest execution

Evaluation result	Timestamp	Target materialization	Partition
Failed	20 feb, 17:35	20 feb, 17:35	—

Metadata

categorias_detectadas	3
categorias_esperadas	2
mensaje	Si hay nombres inconsistentes, ggplot creará leyendas duplicadas.
principio_gestalt	Similitud (Evitar fragmentación visual)

Figura 1.2: Resultado de la materialización con el check fallido.

1.4. Matriz de calidad de checks implementados

La siguiente tabla recoge los nueve checks implementados (3 por cada .py del proyecto, para cada etapa de carga, transformación y visualización) siguiendo la estructura de la matriz de calidad de las diapositivas, adaptada al proyecto de rentas y educación en Canarias:

Etapas	Nombre del Check	Descripción Técnica	Cómo programarlo (Lógica)	Relación con Diseño / Gestalt	Metadata
Carga (Raw)	check_carga_renta	Verifica que el CSV de renta tenga suficientes filas y sin nulos en columnas clave.	<code>len(df) > 1000 and df[['OBS_VALUE', 'TERRITORIO_CODE']].isna().sum().sum() == 0</code>	Figura y Fondo: los huecos en datos clave rompen la serie temporal del gráfico de líneas.	filas, nulos_criticos
Carga (Raw)	check_carga_codislas	Comprueba que el catálogo de islas tiene las columnas CPRO, CMUN, ISLA y NOMBRE.	<code>{'CPRO', 'CMUN', 'ISLA', 'NOMBRE'} - set(df.columns) == set()</code>	Similitud: sin el mapping correcto de islas, categorías distintas se pintarían con el mismo color.	columnas_faltantes
Carga (Raw)	check_carga_estudios	Valida que el Excel de niveles de estudios contiene las columnas necesarias para la transformación.	<code>{'Municipios...', 'Nivel de estudios...', 'Total', 'Periodo'} - set(df.columns) == set()</code>	Figura y Fondo: columnas faltantes generarían NaN que producen barras vacías en el gráfico.	columnas_faltantes, filas
Transf. (Curated)	check_transform_renta	Detecta valores negativos en OBS_VALUE y verifica que hay exactamente 5 medidas únicas.	<code>(df['OBS_VALUE'] < 0).sum() == 0 and df['MEDIDA_CORTA'].nunique() == 5</code>	Similitud: un valor negativo crea una línea anómala que rompe la coherencia visual de la serie.	negativos, medidas_unicas
Transf. (Curated)	check_merge_islas	Comprueba que al menos el 90% de los municipios tienen isla asignada tras el merge con codislas.	<code>(df['ISLA'].notna().sum() / len(df)) * 100 >= 90</code>	Continuidad: municipios sin isla quedan excluidos del gráfico, creando series incompletas.	total_registros, con_isla, porcentaje_cobertura
Transf. (Curated)	check_transform_estudios	Verifica que la población es positiva y que hay exactamente 5 niveles educativos válidos.	<code>(df['POBLACION'] < 0).sum() == 0 and df['NIVEL_CORTO'].nunique() == 5</code>	Similitud / Gestalt: categorías inconsistentes generan barras duplicadas o desiguales entre islas.	poblacion_negativa, niveles_detectados, niveles_esperados
Visualiz. (Asset)	check_viz_paleta_renta	Limita el número de categorías de medida a 8 para que la paleta cualitativa Set1 sea legible.	<code>df['MEDIDA_CORTA'].nunique() <= 8</code>	Carga Cognitiva / Similitud: más de 8 colores son imposibles de distinguir en una leyenda.	categorias, limite
Visualiz. (Asset)	check_viz_top5	Confirma que el número de municipios en leyenda no supera los 8 colores de la paleta Set1.	<code>df['TERRITORIO_CODE'].nunique() <= 8</code>	Carga Cognitiva: una leyenda con más de 8 entradas obliga al usuario a buscar cada línea.	municipios_en_leyenda, max_colores_paleta
Visualiz. (Asset)	check_viz_barras_isla	Verifica que el número de barras agrupadas por isla no supera 6, límite de legibilidad de Set2.	<code>df['NIVEL_CORTO'].nunique() <= 6</code>	Pragnanz: más de 6 barras agrupadas por isla dificultan la comparación y saturan el eje X.	niveles_en_grafico, max_recomendado

Cuadro 1.1: Matriz de calidad de checks implementados en el proyecto, siguiendo la estructura de la plantilla de las diapositivas.

1.5. Implementación de casos sucios por etapa

Todos los casos sucios se implementan **dentro de la función del check**, nunca en el asset. De este modo el asset siempre se materializa correctamente y solo falla la evaluación de calidad, sin interrumpir el pipeline.

1.5.1. Etapa de Carga

renta_canarias.py — check_carga_renta

```
nulos_criticos = int(df[['OBS_VALUE', 'TERRITORIO_CODE']].isnull()
    .sum().sum())
passed = bool((filas > 1000) and (nulos_criticos == 0))
# Resultado: passed = True
```

Listing 1.1: Check válido: todos los datos están completos.

```
nulos_criticos = nulos_criticos + 999
passed = bool((filas > 1000) and (nulos_criticos == 0))
# Resultado: passed = False -- nulos_criticos = 999
```

Listing 1.2: Caso sucio: se suman 999 nulos ficticios para forzar el fallo.

The screenshot shows the Databricks UI for the 'raw_renta_canarias' asset. The 'Checks' tab is active, displaying a list of checks on the left and details for 'check_carga_renta' on the right. The check is marked as 'Succeeded'. The 'Latest execution' table shows a 'Passed' result at 18:36 on Feb 21. The 'Metadata' section shows 'filas' as 4410 and 'nulos_criticos' as 0.

Check	Status
check_carga_renta	Succeeded

Latest execution								
<table border="1"><thead><tr><th>Evaluation result</th><th>Timestamp</th><th>Target materialization</th><th>Partition</th></tr></thead><tbody><tr><td>Passed</td><td>21 feb, 18:36</td><td>21 feb, 18:36</td><td>—</td></tr></tbody></table>	Evaluation result	Timestamp	Target materialization	Partition	Passed	21 feb, 18:36	21 feb, 18:36	—
Evaluation result	Timestamp	Target materialization	Partition					
Passed	21 feb, 18:36	21 feb, 18:36	—					

Metadata
filas 4410
nulos_criticos 0
principio Integridad datos fuente

Figura 1.3: Resultado de la materialización con el check pasado correctamente.

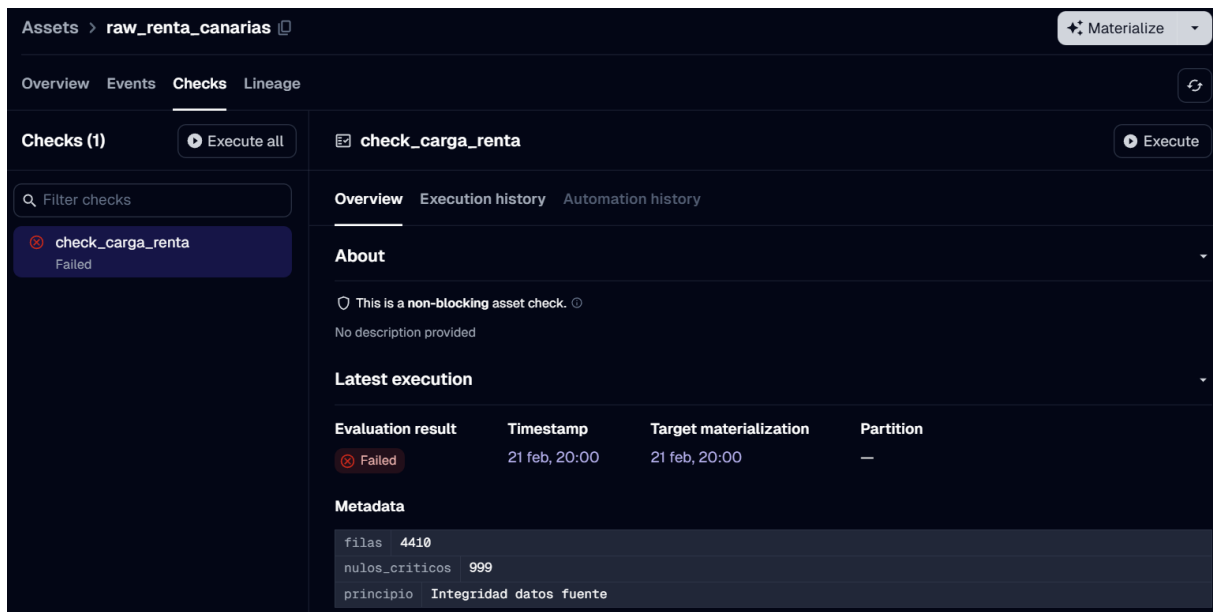


Figura 1.4: Resultado de la materialización con el check fallido.

renta_islas.py — check_carga_codislas

```
faltan = cols_esperadas - set(raw_codislas.columns)
passed = bool(len(faltan) == 0)
# Resultado: passed = True
```

Listing 1.3: Check válido: todas las columnas esperadas están presentes.

```
faltan = faltan | {'ISLA'}
passed = bool(len(faltan) == 0)
# Resultado: passed = False -- columnas_faltantes: ['ISLA']
```

Listing 1.4: Caso sucio: se añade 'ISLA' al conjunto de columnas faltantes.

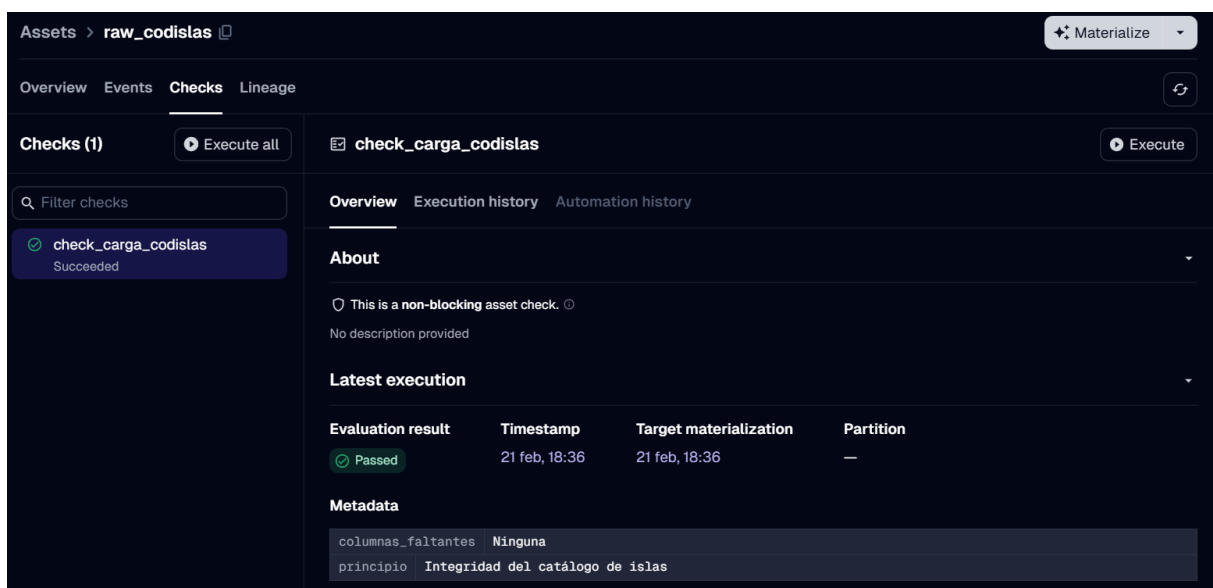


Figura 1.5: Resultado de la materialización con el check pasado correctamente.

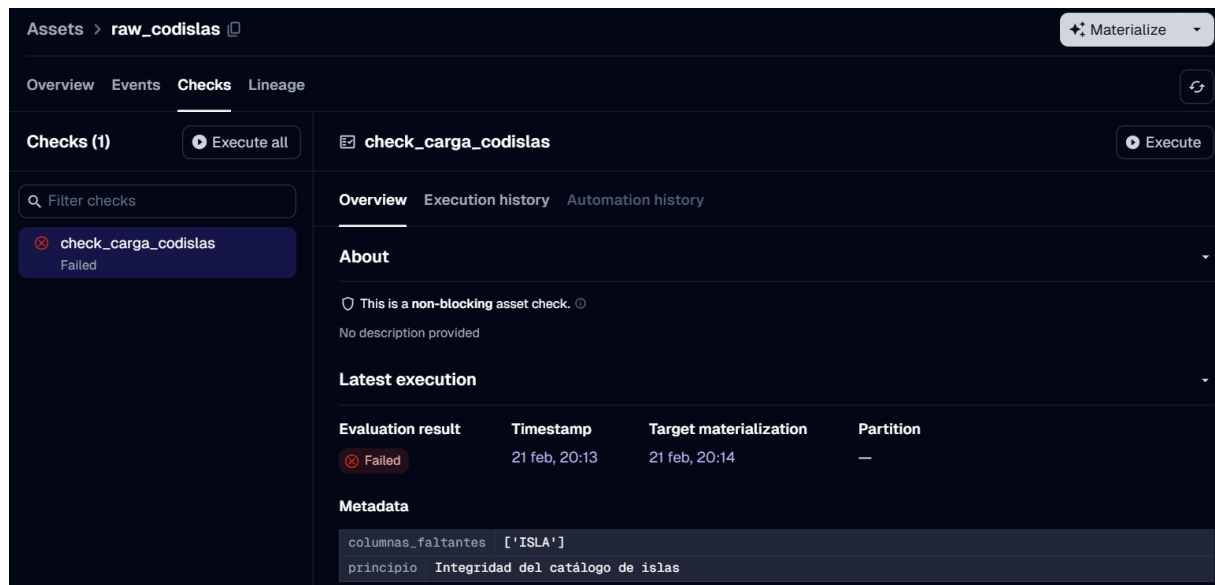


Figura 1.6: Resultado de la materialización con el check fallido.

nivel_estudios.py — check_carga_estudios

```
faltan = cols_esperadas - set(raw_nivel_estudios.columns)
passed = bool(len(faltan) == 0)
# Resultado: passed = True
```

Listing 1.5: Check válido: el Excel contiene todas las columnas requeridas.

```
faltan = faltan | {'Nivel de estudios en curso'}
passed = bool(len(faltan) == 0)
# Resultado: passed = False -- columnas_faltantes: ['Nivel de
estudios en curso']
```

Listing 1.6: Caso sucio: se simula la ausencia de la columna clave de niveles.

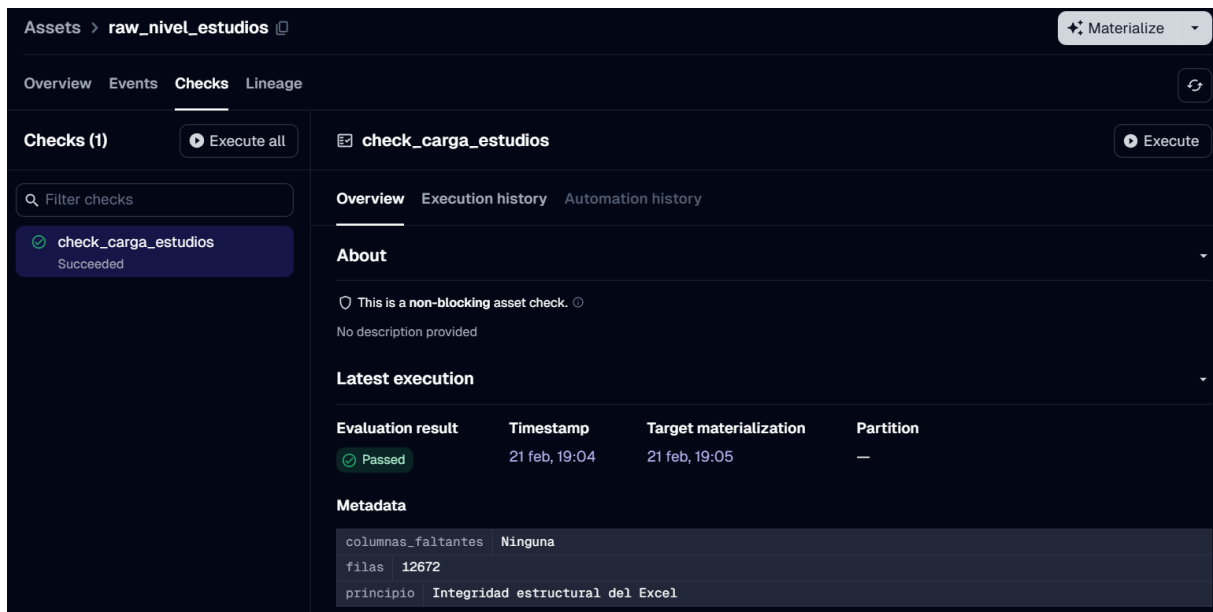


Figura 1.7: Resultado de la materialización con el check pasado correctamente.

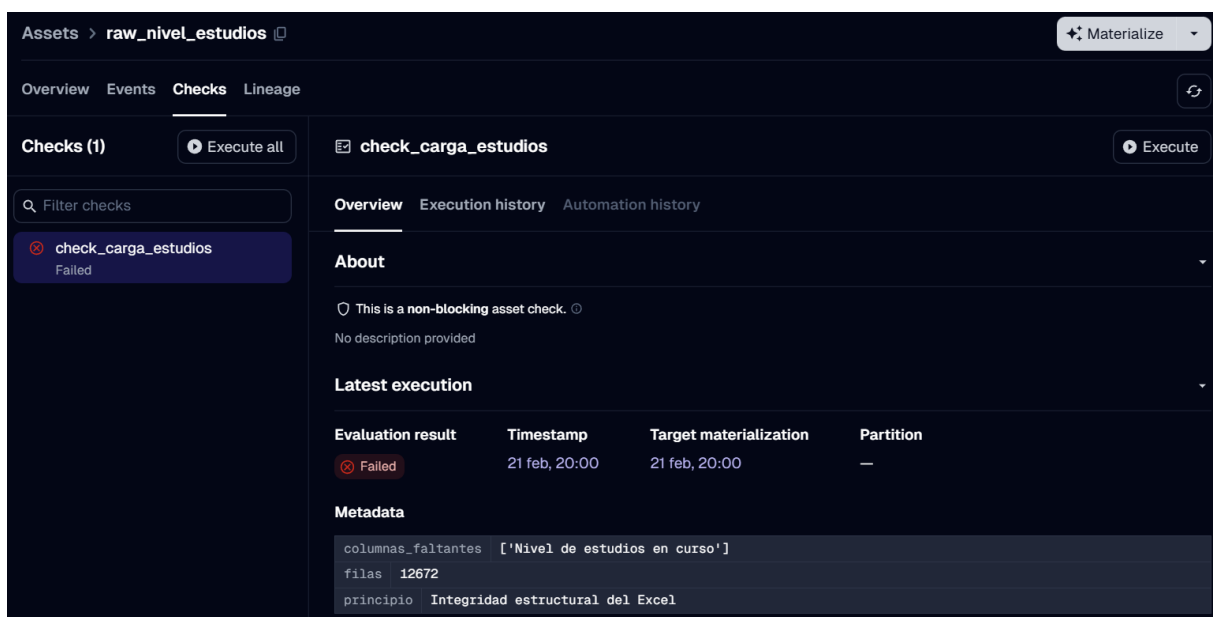


Figura 1.8: Resultado de la materialización con el check fallido.

1.5.2. Etapa de Transformación

renta_canarias.py — check_transform_renta

```
negativos = int((transformed_canarias["OBS_VALUE"] < 0).sum())
passed = bool((negativos == 0) and (medidas == 5))
# Resultado: passed = True
```

Listing 1.7: Check válido: sin valores negativos en OBS_VALUE.

```
negativos = negativos + 10
passed = bool((negativos == 0) and (medidas == 5))
# Resultado: passed = False -- negativos: 10
```

Listing 1.8: Caso sucio: se suman 10 negativos ficticios al conteo.

The screenshot shows the Databricks interface for the asset 'transformed_canarias'. The 'Checks' tab is active, displaying a list of checks on the left and details for 'check_transform_renta' on the right. The check is marked as 'Succeeded' with a green checkmark. The 'Latest execution' table shows a 'Passed' result at 18:36 on Feb 21. The metadata indicates 5 unique measures and 0 negative values.

Check	Status
check_transform_renta	Succeeded

Latest execution	Timestamp	Target materialization	Partition
Passed	21 feb, 18:36	21 feb, 18:37	—

Metadata	Value
medidas_unicas	5
negativos	0
principio	Datos preparados para viz

Figura 1.9: Resultado de la materialización con el check pasado correctamente.

The screenshot shows the Databricks interface for the asset 'transformed_canarias'. The 'Checks' tab is active, displaying a list of checks on the left and details for 'check_transform_renta' on the right. The check is marked as 'Failed' with a red X. The 'Latest execution' table shows a 'Failed' result at 20:00 on Feb 21. The metadata indicates 5 unique measures and 10 negative values.

Check	Status
check_transform_renta	Failed

Latest execution	Timestamp	Target materialization	Partition
Failed	21 feb, 20:00	21 feb, 20:01	—

Metadata	Value
medidas_unicas	5
negativos	10
principio	Datos preparados para viz

Figura 1.10: Resultado de la materialización con el check fallido.

renta_islas.py — check_merge_islas

```
pct = float(con_isla / total * 100) if total > 0 else 0.0
```

```
passed = bool(pct >= 90)
# Resultado: passed = True
```

Listing 1.9: Check válido: más del 90 % de municipios tienen isla asignada.

```
pct = 0.0
passed = bool(pct >= 90)
# Resultado: passed = False -- porcentaje_cobertura: 0.0%
```

Listing 1.10: Caso sucio: se fuerza la cobertura a 0 % dentro del check.

The screenshot shows the Databricks UI for the asset 'municipios_con_islas'. The 'Checks' tab is active, showing a list of checks on the left and details for 'check_merge_islas' on the right. The check is marked as 'Succeeded'.

Checks (1)

- check_merge_islas (Succeeded)

check_merge_islas

Overview Execution history Automation history

About

This is a non-blocking asset check. No description provided.

Latest execution

Evaluation result	Timestamp	Target materialization	Partition
Passed	21 feb, 18:36	21 feb, 18:37	—

Metadata

con_isla	3915
porcentaje_cobertura	100
principio	Cobertura del merge codislas
total_registros	3915

Figura 1.11: Resultado de la materialización con el check pasado correctamente.

The screenshot shows the Databricks UI for the asset 'municipios_con_islas'. The 'Checks' tab is active, showing a list of checks on the left and details for 'check_merge_islas' on the right. The check is marked as 'Failed'.

Checks (1)

- check_merge_islas (Failed)

check_merge_islas

Overview Execution history Automation history

About

This is a non-blocking asset check. No description provided.

Latest execution

Evaluation result	Timestamp	Target materialization	Partition
Failed	21 feb, 20:00	21 feb, 20:01	—

Metadata

con_isla	3915
porcentaje_cobertura	0
principio	Cobertura del merge codislas
total_registros	3915

Figura 1.12: Resultado de la materialización con el check fallido.

nivel_estudios.py — check_transform_estudios

```
negativos = int((cleaned_nivel_estudios['POBLACION'] < 0).sum())
passed = bool((negativos == 0) and (niveles == len(
    NIVELES_VALIDOS)))
# Resultado: passed = True
```

Listing 1.11: Check válido: sin población negativa y exactamente 5 niveles.

```
negativos = negativos + 5
passed = bool((negativos == 0) and (niveles == len(
    NIVELES_VALIDOS)))
# Resultado: passed = False -- poblacion_negativa: 5
```

Listing 1.12: Caso sucio: se suman 5 negativos ficticios al conteo.

The screenshot shows the Databricks Checks interface for the asset 'cleaned_nivel_estudios'. The 'Checks' tab is active, showing a list of checks on the left and details for 'check_transform_estudios' on the right. The check has a status of 'Succeeded'. The 'Latest execution' table shows a 'Passed' result at '21 feb, 18:36'. The 'Metadata' section lists 'niveles_detectados: 5', 'niveles_esperados: 5', 'poblacion_negativa: 0', and 'principio: Similitud Gestalt: categorías consistentes'.

Check	Status
check_transform_estudios	Succeeded

Latest execution								
<table border="1"><thead><tr><th>Evaluation result</th><th>Timestamp</th><th>Target materialization</th><th>Partition</th></tr></thead><tbody><tr><td>Passed</td><td>21 feb, 18:36</td><td>21 feb, 18:36</td><td>—</td></tr></tbody></table>	Evaluation result	Timestamp	Target materialization	Partition	Passed	21 feb, 18:36	21 feb, 18:36	—
Evaluation result	Timestamp	Target materialization	Partition					
Passed	21 feb, 18:36	21 feb, 18:36	—					

Metadata
niveles_detectados: 5
niveles_esperados: 5
poblacion_negativa: 0
principio: Similitud Gestalt: categorías consistentes

Figura 1.13: Resultado de la materialización con el check pasado correctamente.

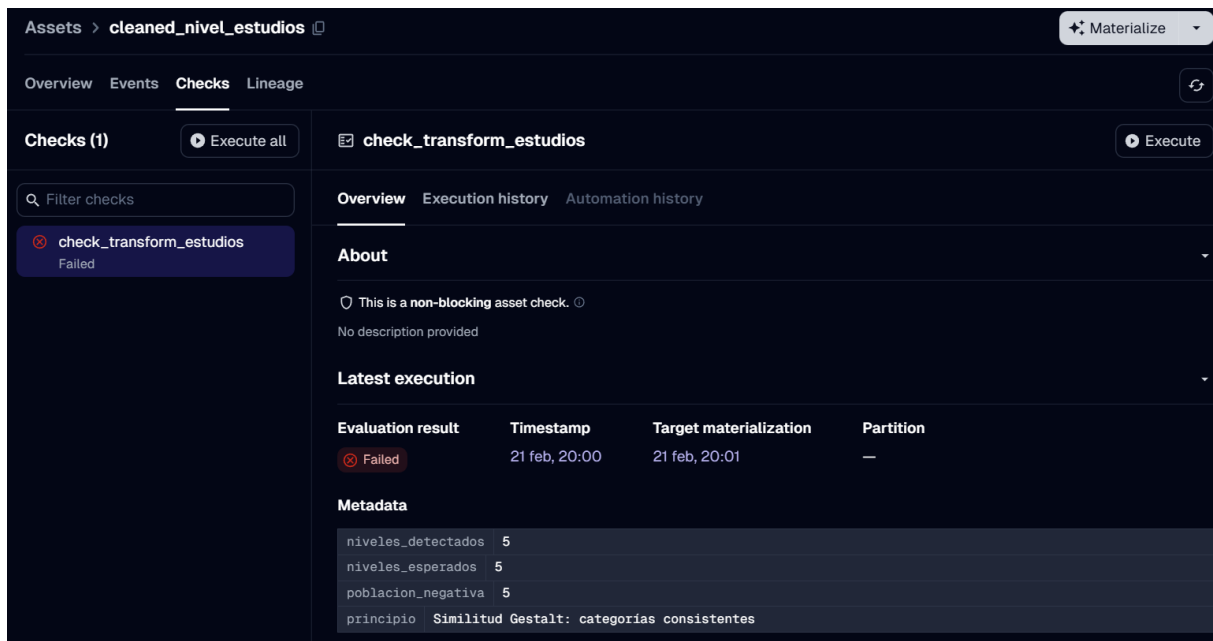


Figura 1.14: Resultado de la materialización con el check fallido.

1.5.3. Etapa de Visualización

renta_canarias.py — check_viz_paleta_renta

```
categorias = int(df['MEDIDA_CORTA'].nunique())
passed = bool(categorias <= 8)
# Resultado: passed = True -- categorias: 5
```

Listing 1.13: Check válido: 5 categorías reales, dentro del límite de 8.

```
extras = pd.DataFrame({'MEDIDA_CORTA': ['Extra1', 'Extra2', 'Extra3', 'Extra4']})
df = pd.concat([df, extras], ignore_index=True)
categorias = int(df['MEDIDA_CORTA'].nunique()) # -> 9
passed = bool(categorias <= 8)
# Resultado: passed = False -- categorias: 9
```

Listing 1.14: Caso sucio: se añaden 4 categorías ficticias al DataFrame local.

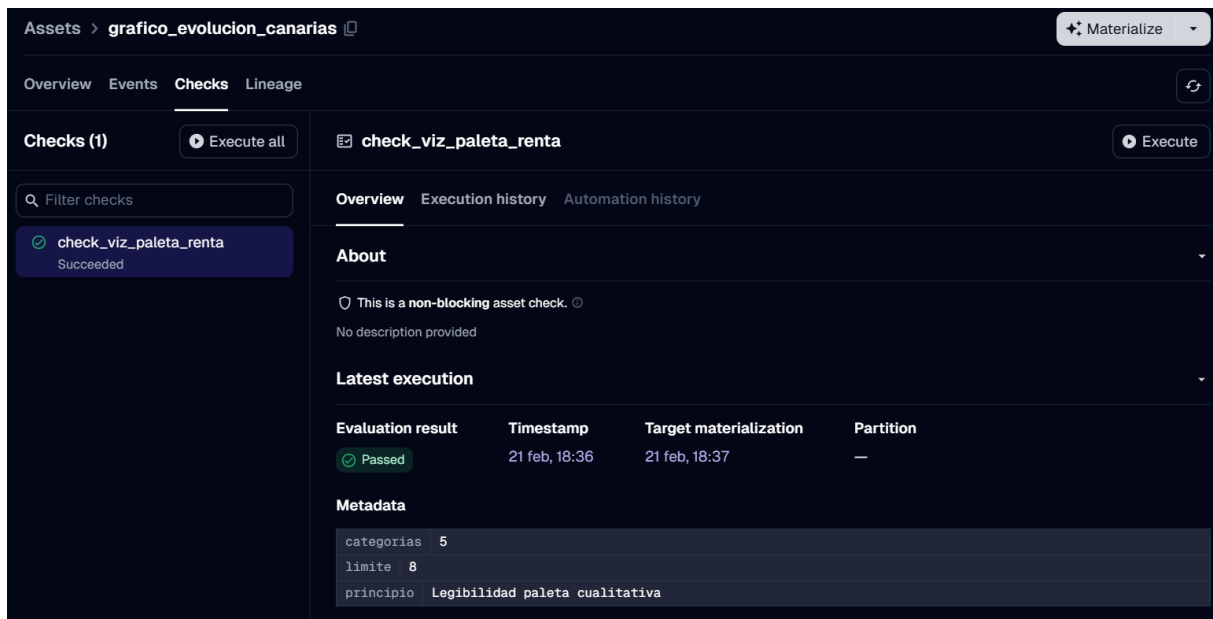


Figura 1.15: Resultado de la materialización con el check pasado correctamente.

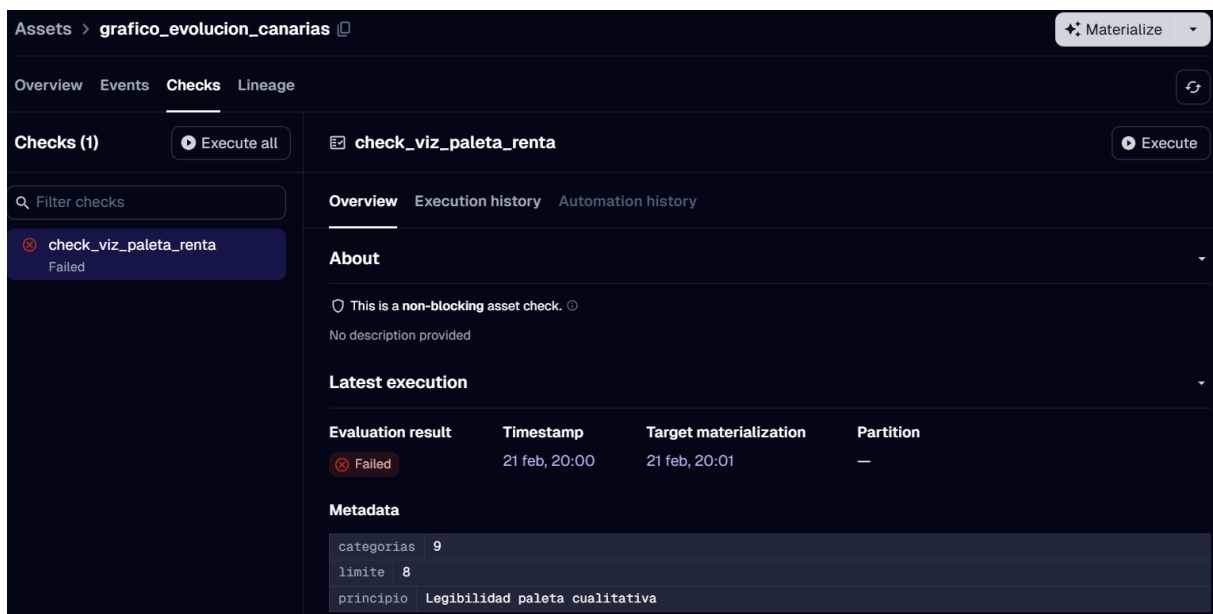


Figura 1.16: Resultado de la materialización con el check fallido.

renta_islas.py — check_viz_top5

```
municipios = int(df_top['TERRITORIO_CODE'].nunique())
passed = bool(municipios <= 8)
# Resultado: passed = True -- municipios: 5
```

Listing 1.15: Check válido: 5 municipios reales, dentro del límite de 8.

```
extras = pd.DataFrame({'TERRITORIO_CODE':
    ['99001', '99002', '99003', '99004', '99005']})
df_top = pd.concat([df_top, extras], ignore_index=True)
```

```

municipios = int(df_top['TERRITORIO_CODE'].nunique()) # -> 10
passed = bool(municipios <= 8)
# Resultado: passed = False -- municipios: 10

```

Listing 1.16: Caso sucio: se añaden 5 municipios ficticios al conteo local.

The screenshot shows the Databricks Checks interface for the asset 'grafico_top5_evolucion_medidas'. The 'Checks' tab is active, showing a list of checks on the left and details for 'check_viz_top5' on the right. The check is marked as 'Succeeded' (green checkmark). The 'Latest execution' table shows a 'Passed' result at 21 feb, 18:36. The metadata indicates 'max_colores_paleta: 8' and 'municipios_en_leyenda: 5'.

Check	Status
check_viz_top5	Succeeded

Latest execution	Timestamp	Target materialization	Partition
Passed	21 feb, 18:36	21 feb, 18:37	—

Metadata	Value
max_colores_paleta	8
municipios_en_leyenda	5
principio	Paleta Set1 legible con máx. 8 series

Figura 1.17: Resultado de la materialización con el check pasado correctamente.

The screenshot shows the Databricks Checks interface for the asset 'grafico_top5_evolucion_medidas'. The 'Checks' tab is active, showing a list of checks on the left and details for 'check_viz_top5' on the right. The check is marked as 'Failed' (red X). The 'Latest execution' table shows a 'Failed' result at 21 feb, 20:29. The metadata indicates 'max_colores_paleta: 8' and 'municipios_en_leyenda: 10'.

Check	Status
check_viz_top5	Failed

Latest execution	Timestamp	Target materialization	Partition
Failed	21 feb, 20:29	21 feb, 20:30	—

Metadata	Value
max_colores_paleta	8
municipios_en_leyenda	10
principio	Paleta Set1 legible con máx. 8 series

Figura 1.18: Resultado de la materialización con el check fallido.

nivel_estudios.py — check_viz_barras_isla

```

niveles = int(df[df['NIVEL_CORTO'].isin(NIVELES_VALIDOS)][['NIVEL_CORTO']].nunique())

```



```
passed = bool(niveles <= 6)
# Resultado: passed = True -- niveles: 5
```

Listing 1.17: Check válido: 5 niveles reales, dentro del límite de 6.

```
extras = pd.DataFrame({'NIVEL_CORTO': ['Doctorado', 'Formacion_Extra', 'Sin_Estudios', 'Formacion_post_doc', 'certificacion_profesional']})
df = pd.concat([df, extras], ignore_index=True)
niveles = int(df[df['NIVEL_CORTO'].isin(NIVELES_VALIDOS + ['Doctorado', 'Formacion_Extra', 'Sin_Estudios', 'Formacion_post_doc', 'certificacion_profesional'])['NIVEL_CORTO'].nunique()) # -> 10
passed = bool(niveles <= 6)
# Resultado: passed = False -- niveles: 10
```

Listing 1.18: Caso sucio: se añaden 5 niveles ficticios superando el límite.

The screenshot shows the Databricks Checks interface for the asset 'viz_educacion_isla_2023'. The 'Checks (1)' tab is active, showing a single check named 'check_viz_barras_isla' with a status of 'Succeeded'. The 'Overview' tab for this check is selected, displaying the following information:

- About:** This is a non-blocking asset check. No description provided.
- Latest execution:**

Evaluation result	Timestamp	Target materialization	Partition
Passed	21 feb, 18:36	21 feb, 18:37	—
- Metadata:**

max_recomendado	6
niveles_en_grafico	5
principio	Legibilidad barras agrupadas: máx. 6 grupos por isla

Figura 1.19: Resultado de la materialización con el check pasado correctamente.

Assets > viz_educacion_isla_2023 Materialize

Overview Events **Checks** Lineage

Checks (1) Execute all

check_viz_barras_isla
Failed

check_viz_barras_isla Execute

Overview Execution history Automation history

About

This is a **non-blocking** asset check. ⓘ
No description provided

Latest execution

Evaluation result	Timestamp	Target materialization	Partition
Failed	21 feb, 21:29	21 feb, 21:31	—

Metadata

max_recomendado	6
niveles_en_grafico	10
principio	Legibilidad barras agrupadas: máx. 6 grupos por isla

Figura 1.20: Resultado de la materialización con el check fallido.

Bibliografía

- [1] Lasso García, L. (2026). *Repositorio del Proyecto: Análisis de Renta y Educación en Canarias (Rama: practica-calidad-checks)*. GitHub. <https://github.com/LauraLasso/dagster-canarias-analytics/tree/practica-calidad-checks>
- [2] Instituto Canario de Estadística (ISTAC). *Distribución de la renta personal bruta de los hogares por municipios de Canarias*. Recuperado de: https://www3.gobiernodecanarias.org/istac/statistical-visualizer/visualizer/collection.html?resourceType=collection&agencyId=ISTAC&resourceId=E30325A_000001
- [3] Dagster Labs. (2026). *Dagster Documentation: Asset Checks and Data Quality*. <https://docs.dagster.io/concepts/assets/asset-checks>
- [4] Hassan K. (2024). *Plotnine: A Grammar of Graphics for Python*. <https://plotnine.org/>
- [5] Wertheimer, M. (1923). *Laws of Organization in Perceptual Forms (Gestalt Theory)*. Explicación aplicada a la visualización de datos.
- [6] Eremin, N., & Otros. (2023). *DataOps Principles: Continuous Integration and Quality Control in Data Pipelines*.